AI Assisted Coding

Lab Assignment 15.5

Name : K. Charan Yadav

Hall Ticket no : 2303A52367

Batch No : 20

Task -1:

Prompt: The task is to develop a Student Attendance RESTful API using Node.js and Express. The API should support endpoints for viewing attendance records, marking attendance, updating attendance records, and deleting attendance entries. The API must handle JSON-based requests and responses. The system should store attendance data in memory and allow CRUD operations through HTTP methods like GET, POST, PUT, and DELETE.

```
Assignment-14.1.html   Assignment-15.5.js   Assignment-16.3.py   Assignment-17.5.py
Assignment-15.5.js > ...
# Task -1:
// Import Express
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// In-memory storage
let attendanceRecords = [];
let nextId = 1;

// Root route
app.get('/', (req, res) => {
    res.status(200).send('Attendance API is running');
});

// GET all attendance records
app.get('/attendance', (req, res) => {
    res.status(200).json(attendanceRecords);
});

// POST add attendance record
app.post('/attendance', (req, res) => {
    const { studentId, date, status } = req.body;

    if (!studentId || !date || !status) {
        return res.status(400).json({
            message: 'Missing required fields: studentId, date, and status are required'
        });
    }

    const normalizedStatus = status.toString().trim().toLowerCase();

    if (normalizedStatus !== 'present' && normalizedStatus !== 'absent') {
        return res.status(400).json({
            message: "Status must be either 'present' or 'absent'"
        });
    }

    const newRecord = {
        id: nextId++,
        studentId,
        date,
        status: normalizedStatus
    };

    attendanceRecords.push(newRecord);
    res.status(201).json(newRecord);
});
```

OUTPUT:

```
Pretty-print
[{"id":1,"studentId":"101","date":"2026-04-01","status":"present"},{"id":3,"studentId":"102","date":"2026-04-01","status":"absent"},{"id":4,"studentId":"104","date":"2026-04-
01","status":"absent"}]
```

Justification: This project demonstrates how RESTful APIs can be built using Node.js and Express for managing student attendance records. It helps in understanding CRUD operations, JSON data handling, and API endpoint development. The system allows easy maintenance and updating of attendance records through HTTP requests. This project is useful for learning backend development and API design concepts.

Task 2:

Prompt: The task is to develop a REST API for warehouse inventory management using Node.js and Express. The API should provide endpoints to list all inventory items, add new items, update stock quantity, and delete items. The system should validate stock quantity and price values before updating the inventory. The API must handle JSON-based requests and responses for inventory operations.

Assignment-14.3.html | Assignment-15.5.js 9+ | Assignment-16.3.py | Assignment-17.5.py

Assignment-15.5.js > ...

```javascript
#Task-2 :
// Import Express
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Inventory storage
let inventory = [];

// Home route
app.get('/', (req, res) => {
    res.send("Inventory API is running...");
});

// GET /inventory → List all items
app.get('/inventory', (req, res) => {
    res.json(inventory);
});

// POST /inventory → Add new item
app.post('/inventory', (req, res) => {
    const { itemName, quantity, price } = req.body;

    // Validation
    if (!itemName || quantity === undefined || price === undefined) {
        return res.status(400).json({
            message: "All fields are required"
        });
    }

    if (quantity < 0 || price <= 0) {
        return res.status(400).json({
            message: "Invalid stock or price value"
        });
    }

    const newItem = {
        id: inventory.length + 1,
        itemName,
        quantity,
        price
    };

    inventory.push(newItem);
    res.status(201).json(newItem);
});

// PUT /inventory/:id → Update stock quantity
app.put('/inventory/:id', (req, res) => {
    const itemId = parseInt(req.params.id);
    const { quantity } = req.body;

```

Output:

Pretty-print

[{"id":1,"itemName":"Laptop","quantity":10,"price":50000},{"id":2,"itemName":"mouse","quantity":10,"price":5000},{"id":3,"itemName":"cpu","quantity":10,"price":30000}]

Justification: This project demonstrates how REST APIs can be used to manage warehouse inventory and stock levels efficiently. It helps in understanding CRUD operations, data validation, and API development using Node.js and Express. The system ensures that invalid stock values are not entered through validation. This project is useful for learning backend development and inventory management systems.

Task 3:

Prompt: The task is to develop a RESTful API for hotel room booking using Node.js and Express. The API should allow users to view available rooms, book a room, view booking details by room ID, and cancel bookings. The system should handle errors such as booking unavailable rooms or invalid room IDs. The API must use JSON-based request and response handling.

Assignment-14.3.html | Assignment-15.5.js 9+ | Assignment-16.3.py | Assignment-17.5.py

Assignment-15.5.js > ...

```javascript
# Task-3 :
// Import Express
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Sample rooms data
let rooms = [
    { id: 1, roomNumber: "101", type: "Single", price: 1000, isBooked: false, customer: null },
    { id: 2, roomNumber: "102", type: "Double", price: 2000, isBooked: false, customer: null },
    { id: 3, roomNumber: "103", type: "Suite", price: 4000, isBooked: false, customer: null },
    { id: 4, roomNumber: "104", type: "Single", price: 1200, isBooked: false, customer: null }
];

// Home route
app.get('/', (req, res) => {
    res.send("Hotel Booking API is running...");
});

// GET /rooms → View available rooms
app.get('/rooms', (req, res) => {
    const availableRooms = rooms.filter(room => !room.isBooked);
    res.json(availableRooms);
});

// POST /rooms/book → Book a room
app.post('/rooms/book', (req, res) => {
    const { roomId, customerName } = req.body;

    if (!roomId || !customerName) {
        return res.status(400).json({
            message: "roomId and customerName are required"
        });
    }

    const room = rooms.find(r => r.id === roomId);

    if (!room) {
        return res.status(404).json({ message: "Room not found" });
    }

    if (room.isBooked) {
        return res.status(400).json({ message: "Room already booked" });
    }

    room.isBooked = true;
    room.customer = customerName;

    res.status(201).json({
        message: "Room booked successfully",
        room
    });
```

Output:

Pretty-print

[{"id":1,"roomNumber":"101","type":"Single","price":1000,"isBooked":false,"customer":null},{"id":2,"roomNumber":"102","type":"Double","price":2000,"isBooked":false,"customer":null},{"id":3,"roomNumber":"103","type":"Suite","price":4000,"isBooked":false,"customer":null},{"id":4,"roomNumber":"104","type":"Single","price":1200,"isBooked":false,"customer":null}]

Justification:

This project demonstrates how RESTful APIs can be used to simulate hotel room booking operations. It helps in understanding API routing, booking logic, and error handling in backend development. The system ensures that rooms cannot be double booked and handles invalid booking requests. This project is useful for learning real-world booking system API development using Node.js and Express.

Task -4:

Prompt: The task is to create a RESTful API for an online voting system using Node.js and Express. The API should allow users to view the list of candidates, cast a vote, and view voting results. The system must ensure that each user can vote only once using in-memory validation. The API should handle JSON-based requests and responses for voting operations.

```
Assignment-14.3.html   Assignment-15.5.js 9+   Assignment-16.3.py   Assignment-17.5.py
Assignment-15.5.js > ...
# Task-4 :
// Import Express
const express = require('express');
const app = express();

// Middleware
app.use(express.json());

// In-memory data (Candidates)
let candidates = [
    { id: 1, name: "Alice", votes: 0 },
    { id: 2, name: "Bob", votes: 0 },
    { id: 3, name: "Charlie", votes: 0 }
];

// Store users who have voted
let votedUsers = new Set();

// Home route
app.get('/', (req, res) => {
    res.send("Online Voting API is running...");
});

// GET /candidates → List candidates
app.get('/candidates', (req, res) => {
    res.json(candidates);
});

// POST /vote → Cast a vote
app.post('/vote', (req, res) => {
    const { userId, candidateId } = req.body;

    // Validate input
    if (!userId || !candidateId) {
        return res.status(400).json({
            message: "userId and candidateId are required"
        });
    }

    // Check if user already voted
    if (votedUsers.has(userId)) {
        return res.status(403).json({
            message: "User has already voted"
        });
    }

    // Find candidate
    const candidate = candidates.find(c => c.id === candidateId);

    if (!candidate) {
        return res.status(404).json({
            message: "Candidate not found"
        });
    }
```

Output:

```
Pretty-print
[{"id":1,"name":"Alice","votes":0},{"id":2,"name":"Bob","votes":0},{"id":3,"name":"Charlie","votes":0}]
```

Justification:

This project demonstrates how RESTful APIs can be used to implement an online voting system with basic security validation. It helps in understanding API development, data validation, and preventing duplicate voting using in-memory storage. The system simulates a secure voting process and result calculation. This project is useful for learning backend API development and voting system logic.

Task – 5:

Prompt: The task is to develop a RESTful API to manage teacher records using Node.js and Express. The API should allow users to list all teachers, add new teacher records, update teacher details, and delete teacher records. The system must handle JSON-based requests and responses and include proper error handling for invalid teacher IDs or missing data. The API should support CRUD operations for teacher management.

```
Assignment-14.3.html   Assignment-15.5.js 9+   Assignment-16.3.py   Assignmen
Assignment-15.5.js > ...

# Task - 5:
// Import Express
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// In-memory data (Candidates)
let candidates = [
    { id: 1, name: "Alice", votes: 0 },
    { id: 2, name: "Bob", votes: 0 },
    { id: 3, name: "Charlie", votes: 0 }
];

// Store users who have voted
let votedUsers = new Set();

// Home route
app.get('/', (req, res) => {
    res.send("Online Voting API is running...");
});

// GET /candidates → List candidates
app.get('/candidates', (req, res) => {
    res.json(candidates);
});

// POST /vote → Cast a vote
app.post('/vote', (req, res) => {
    const { userId, candidateId } = req.body;

    // Validate input
    if (!userId || !candidateId) {
        return res.status(400).json({
            message: "userId and candidateId are required"
        });
    }

    // Check if user already voted
    if (votedUsers.has(userId)) {
        return res.status(403).json({
            message: "User has already voted"
        });
    }

    // Find candidate
    const candidate = candidates.find(c => c.id === candidateId);

    if (!candidate) {
        return res.status(404).json({
            message: "Candidate not found"
        });
```

Output:

Justification:

This project demonstrates how RESTful APIs can be used to manage teacher information using CRUD operations. It helps in understanding API routing, JSON data handling, and error handling in backend development. The system allows efficient management of teacher records such as adding, updating, and deleting information. This project is useful for learning backend API development and data management systems.